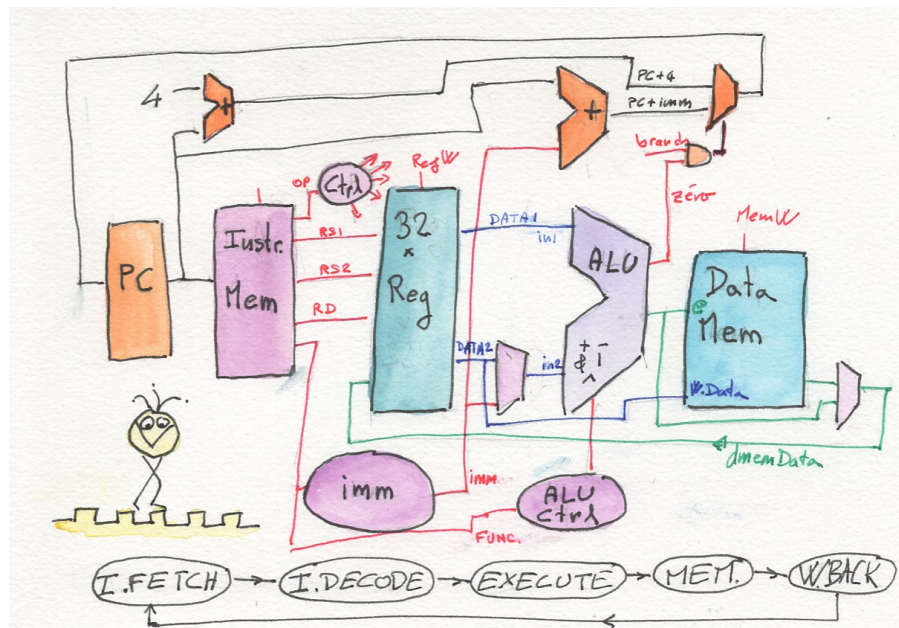


INTRODUCTION À L'ARCHITECTURE DES ORDINATEURS



Chemin de données d'un processeur RISC-V

VOYAGE AU COEUR DU PROCESSEUR RISC-V

Historique des révisions :

2026 : version initiale.

A faire :

Dans

https://github.com/JOKleinGe1/Simplified_RISCV/blob/main/riscv_simple_v2.vhd

retirer le commentaire final et préfixer les signaux de debug en dbg_xx.

Mettre à jour dans ce doc

Créer un nouveau repo propre (sans verilog) dans github.

Tester et rédiger les évaluations.

Présentation

Ce document accompagne les enseignements proposés au semestre 3 du parcours « Électronique et Systèmes Embarqués (ESE) dans le département GEii-1 de l'IUT de Cachan. Il se rapporte aux ressources « **R3.07 : informatique industrielle** », dans son volet « Étude et compréhension de l'architecture des unités de calcul ». Il s'appuie sur la ressource « **R3.ESE.15 : Electronique spécialisée** », notamment le codage en langage de description matérielle et l'implantation sur FPGA. Il s'applique particulièrement dans la situation professionnelle « Implanter une solution matérielle ou logicielle dans une partie ou sous partie d'un système » (cf. PN-GEii page 16).

Acquis d'apprentissage visés (AAV)

A l'issue des 4 séances de ce module d'introduction à l'architecture des ordinateurs, les étudiants seront capables de :

AAV-1 : Prévoir l'effet de l'exécution d'un programme assembleur (RISCV) sur chacun de ses blocs (PC, registres, ALU, mémoires), pour un jeu d'instructions limité (RV32I).

→ **Evaluation : test écrit.**

AAV-2 : Analyser la simulation de l'exécution d'un processeur RISCV, par exemple pour retrouver les instructions assembleurs exécutées à partir d'un chronogramme.

→ **Evaluation : test écrit**

AAV-3 : Modifier un code assembleur fourni pour implémenter une nouvelle fonctionnalité et produire le code binaire pour l'intégrer dans une description en code en HDL. → **Evaluation : test écrit + test TP**

AAV-4 : Réaliser la synthèse sur cible FPGA d'un processeur décrit en HDL.

→ **Evaluation : test TP.**

AAV-5 : Modifier un code HDL fourni pour implémenter et démontrer une nouvelle instruction ou une nouvelle fonctionnalité. → **Evaluation : test TP.**

Ressources

Des ressources logicielles accompagnent ce document. Elles sont disponibles ici :

https://github.com/JOKleinGe1/Simplified_RISCV

Ces ressources s'appuient sur un modèle simplifié de processeur 32 bits : le **RISC-V**. Son jeu d'instructions libre de droit et les implémentations HDL disponibles gratuitement (contrairement aux processeurs ARM ou intel), en font un choix idéal pour un usage pédagogique dans l'enseignement supérieur.

Séance 1

Analyse du code, identification des blocs, analyse de l'exécution d'une instruction, présentation au groupe.

AAV-1 : Prévoir l'effet de l'exécution d'un programme assembleur (RISCV) sur chacun de ses blocs (PC, registres, ALU, mémoires), pour un jeu d'instructions limité (RV32I).

AAV-2 : Analyser la simulation de l'exécution d'un processeur RISCV, par exemple pour retrouver les instructions assembleurs exécutées à partir d'un chronogramme.

Ressources : cinq documents vous sont fournis :

- Le code VHDL d'un processeur RISC-V simplifié. Toutes les instructions ne sont pas implémentées.
- Le schéma bloc correspondant au chemin de données d'un processeur RISC-V, tel qu'il est décrit dans le livre de Hennessy et Patterson « computer organization and design », 5^e édition.
- Une feuille résumant les instructions du RISC-V.
- Le chronogramme de la simulation de l'exécution d'un programme.
- Le code assembleur du programme exécuté.

Consignes 1 : En équipes, analyser le code HDL fourni. Identifier les blocs composant le chemin de données. Identifier les instructions implémentées dans le code HDL. Identifier les instructions exécutées dans le programme dont la trace est fournie.

Livrable intermédiaire : Une présentation de 5 minutes s'appuyant sur un chemin de données qui aura été reproduit sur une grande feuille pour montrer le fonctionnement d'une instruction, choisie en concertation avec un enseignant.

Consignes 2 : A partir du code fourni sur github ou sur Edaplayground, simuler le code HDL et implémenter le processeur et son programme dans le FPGA d'une carte DE10Lite.

<https://www.edaplayground.com/x/XbrS>

https://github.com/JOKleinGe1/Simplified_RISCV

Séance 2

Modification du code source assembleur, test en simulation, puis en synthèse. Démonstration.

AAV-3 : Modifier un code assembleur fourni pour implémenter une nouvelle fonctionnalité et produire le code binaire pour l'intégrer dans une description en code en HDL.

AAV-4 : Réaliser la synthèse sur cible FPGA d'un processeur décrit en HDL.

Ressources :

- Code source assembleur initial sur github :
- Pour compiler du code ASM RISC-V : <https://riscvasm.lucasteske.dev/#>
- Simulation VHDL sur eda playground : <https://www.edaplayground.com/x/XbrS>
- https://github.com/JOKleinGe1/Simplified_RISCV/blob/main/src/

Consignes : En équipe, choisir une nouvelle fonctionnalité. Par exemple simuler une porte logique (ET, OU, NON) entre 2 ou 3 entrées du port d'entrée (inport, adresse 0x400), avec une sortie sur port de sortie (outport, adresse 0x404). En binôme, modifier le code pour implémenter cette fonctionnalité. Utiliser un compilateur ou un assembleur en ligne pour générer le code binaire à intégrer dans le code HDL. Utiliser un simulateur HDL (par exemple ModelSim) pour vérifier la fonctionnalité. Finalement, implémenter votre solution dans un FPGA.

Livrable 1 : Une présentation de 3 minutes montrant les modifications effectuées dans le code et une démonstration de 2 minutes.

Livrable 2 : Une fiche synthétique A4 résumant les étapes de la compilation du code, de la simulation et de la synthèse sur FPGA. Cette fiche, si elle est validée en séance par votre enseignant, sera autorisée en test TP.

Séance 3

Ajout d'une instruction et validation en simulation et synthèse.

AAV-4 : Réaliser la synthèse sur cible FPGA d'un processeur décrit en HDL.

AAV-5 : Modifier un code HDL fourni pour implémenter et démontrer une nouvelle instruction ou une nouvelle fonctionnalité.

Ressources : ### documents vous sont fournis :

Consignes : En équipe, choisir une instruction du processeur RISC-V (RV32I) non implémentée dans le modèle HDL fourni (par exemple SUB) et l'implémenter dans le code HDL, en respectant scrupuleusement le codage du jeu d'instructions d'un RISC-V RV32I, de façon à ce que le code binaire de l'instruction puisse être compatible avec celui généré par un assembleur ou un compilateur.

Ajouter cette instruction dans votre code binaire et vérifier que l'instruction fonctionne comme prévu. Dans la mesure du possible, vérifier que le fonctionnement des autres instructions n'est pas altéré.

Evaluation écrite des AAV 1 à 3 et évaluation pratique des AAV 3 à 5.

Evaluation écrite :

Exercice É1 : (AAV1) Un code en assembleur RISC-V (RV32I) d'une dizaine de lignes vous est fourni. Vous devez prévoir l'état des registres et des mémoires qui seraient modifiés par l'exécution de ce code.

Exercice É2 : (AAV2) Le chronogramme associé à l'exécution d'un programme assembleur RISC-V (RV32I) vous est fourni. Vous devez retrouver les instructions exécutées et reconstituer le programme complet.

Exercice É3 : (AAV3) Un programme en assembleur et un cahier des charges vous est fourni. Vous devez adapter le programme assembleur pour qu'il réponde au cahier des charges. Par exemple, ce programme pourrait servir à recopier une zone mémoire, ou mettre à zéro une zone mémoire, ou attendre un état spécifique d'un port d'entrée.

Evaluation pratique :

Exercice P1 : (AAV3 et 4) Idem exercice 3 de l'évaluation écrite mais en simulation et synthèse sur cible FPGA.

Exercice P2 : (AAV4 et 5) Implémenter une instruction manquante dans le code HDL fourni sur github. Simuler la puis programmer le FPGA pour démontrer sa fonctionnalité. L'instruction sera choisie par l'enseignant.

Annexe 1

Modèle VHDL de RISC-V simplifié (certaines instructions sont manquantes)

https://github.com/JOKleinGe1/Simplified_RISC-V/blob/main/riscv_simple_v2.vhd

```
-- =====
-- riscv_simple.vhd
-- Traduction VHDL du processeur RISC-V simplifié
-- =====

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity riscv_simple is
  port (
    clk      : in  std_logic;
    reset    : in  std_logic;
    inport   : in  std_logic_vector(7 downto 0);
    outport  : out std_logic_vector(7 downto 0);
    pc_dbg   : out std_logic_vector(23 downto 0)
  );
end entity riscv_simple;

architecture rtl of riscv_simple is

  -- =====
  -- TYPES MEMOIRES
  -- =====
  type imem_t is array (0 to 255) of std_logic_vector(31 downto 0);
  type dmem_t is array (0 to 255) of std_logic_vector(31 downto 0);
  type regfile_t is array (0 to 31) of std_logic_vector(31 downto 0);

  -- =====
  -- ETATS
  -- =====
  constant FETCH : integer := 0;
  constant DECODE : integer := 1;
  constant EXECUTE : integer := 2;
  constant MEM : integer := 3;
  constant WB : integer := 4;
  constant TRAP : integer := 5;
  signal state : integer;

  -- type state_t is (FETCH, DECODE, EXECUTE, MEM, WB, TRAP);
  -- signal state : state_t;

  -- =====
  -- CPU
  -- =====
  signal PC : std_logic_vector(31 downto 0);
  signal instruction : std_logic_vector(31 downto 0);

  signal imem : imem_t := (
    X"00000093",
    X"00500113",
    X"20000193",
    X"00000293",
    X"0011a023",
    X"0001a203",
    X"00108093",
    X"00418193",
    X"004282b3",
    X"00208463",
    X"fe0004e3",
    -- ... (other instructions)
  );
```



```

X"40000193",
X"0001a203",
X"00f24093",
X"0011a223",
X"fe0008e3",
others => X"00000000" );

-- =====
-- BANC DE REGISTRES
-- =====
signal registers : regfile_t := (others => X"00000000");
signal registers_all : std_logic_vector(1023 downto 0);

-- =====
-- MEMOIRE DE DONNEES
-- =====
signal dmem      : dmem_t := (others => X"00000000");
signal dmem_data : std_logic_vector(31 downto 0);
signal dmem_all  : std_logic_vector(8191 downto 0);

-- =====
-- DECODE
-- =====
signal opcode : std_logic_vector(6 downto 0);
signal rd     : std_logic_vector(4 downto 0);
signal funct3 : std_logic_vector(2 downto 0);
signal rs1    : std_logic_vector(4 downto 0);
signal rs2    : std_logic_vector(4 downto 0);

signal read_data1 : std_logic_vector(31 downto 0);
signal read_data2 : std_logic_vector(31 downto 0);

-- =====
-- IMMEDIATS
-- =====
signal imm_i : std_logic_vector(31 downto 0);
signal imm_s : std_logic_vector(31 downto 0);
signal imm_b : std_logic_vector(31 downto 0);
signal imm    : std_logic_vector(31 downto 0);

-- =====
-- SIGNAUX DE CONTROLE
-- =====
signal reg_write : std_logic;
signal alu_src   : std_logic;
signal branch    : std_logic;
signal mem_read  : std_logic;
signal mem_write : std_logic;
signal mem_to_reg : std_logic;

-- =====
-- ALU
-- =====
signal alu_ctrl  : std_logic_vector(3 downto 0);
signal alu_in2   : std_logic_vector(31 downto 0);
signal alu_result : std_logic_vector(31 downto 0);

-- =====
-- BRANCH
-- =====
signal zero : std_logic;

begin
-- pour que tb recupere les registres
g1:for i in 0 to 31 generate
    registers_all(31+(32*i) downto 32*i) <= registers(i);
end generate;

-- pour que le tb recupere dmem

```

```

g2: for i in 0 to 255 generate
    dmem_all(31+(32*i) downto 32*i) <= dmem(i);
end generate;

-- =====
-- DECODE (signaux combinatoires issus de l'instruction)
-- =====
opcode <= instruction(6 downto 0);
rd      <= instruction(11 downto 7);
funct3 <= instruction(14 downto 12);
rs1     <= instruction(19 downto 15);
rs2     <= instruction(24 downto 20);

-- =====
-- IMMEDIATS
-- =====
imm_i <= (31 downto 12 => instruction(31)) & instruction(31 downto 20);

imm_s <= (31 downto 12 => instruction(31)) &
        instruction(31 downto 25) &
        instruction(11 downto 7);

imm_b <= (31 downto 13 => instruction(31)) &
        instruction(31) &
        instruction(7) &
        instruction(30 downto 25) &
        instruction(11 downto 8) &
        '0';

imm <= imm_b when opcode = "1100011" else
        imm_s when opcode = "0100011" else
        imm_i;

-- =====
-- SIGNAUX DE CONTROLE
-- =====
reg_write <= '1' when (opcode = "0110011" or
                      opcode = "0010011" or
                      opcode = "0000011") else '0';

alu_src    <= '1' when (opcode = "0010011" or
                      opcode = "0000011" or
                      opcode = "0100011") else '0';

branch     <= '1' when opcode = "1100011" else '0';
mem_read   <= '1' when opcode = "0000011" else '0';
mem_write  <= '1' when opcode = "0100011" else '0';
mem_to_reg <= '1' when opcode = "0000011" else '0';

-- =====
-- DEBUG PC
-- =====
pc_dbg <= PC(23 downto 0);

-- =====
-- ALU CONTROL (combinatoire)
-- =====
process(opcode, funct3)
begin
    case opcode is
        -- R-TYPE
        when "0110011" =>
            case funct3 is
                when "000" => alu_ctrl <= "0000"; -- ADD
                when "111" => alu_ctrl <= "0001"; -- AND
                when "110" => alu_ctrl <= "0010"; -- OR
                when "100" => alu_ctrl <= "0011"; -- XOR
                when others => alu_ctrl <= "0000";
            end case;
    end case;
end process;

```

```

-- I-TYPE
when "0010011" =>
    case funct3 is
        when "000" => alu_ctrl <= "0000"; -- ADDI
        when "100" => alu_ctrl <= "0011"; -- XORI
        when others => alu_ctrl <= "0000";
    end case;
-- LOAD / STORE
when "0000011" | "0100011" =>
    alu_ctrl <= "0000";
when others =>
    alu_ctrl <= "0000";
end case;
end process;

-- =====
-- ALU (combinatoire)
-- =====
alu_in2 <= imm when alu_src = '1' else read_data2;

process(alu_ctrl, read_data1, alu_in2)
begin
    case alu_ctrl is
        when "0000" =>
            alu_result <= std_logic_vector(
                unsigned(read_data1) + unsigned(alu_in2));
        when "0001" =>
            alu_result <= read_data1 and alu_in2;
        when "0010" =>
            alu_result <= read_data1 or alu_in2;
        when "0011" =>
            alu_result <= read_data1 xor alu_in2;
        when others =>
            alu_result <= (others => '0');
    end case;
end process;

-- =====
-- BRANCH : comparaison pour BEQ
-- =====
zero <= '1' when read_data1 = read_data2 else '0';

-- =====
-- MACHINE D'ETAT (synchrone)
-- =====
process(clk)
begin
    if rising_edge(clk) then

        if reset = '0' then
            PC <= (others => '0');
            state <= FETCH;

        else
            case state is

                -- -----
                -- FETCH
                -- -----
                when FETCH =>
                    instruction <= imem(to_integer(unsigned(PC(9 downto 2))));
                    state <= DECODE;

                -- -----
                -- DECODE
                -- -----
                when DECODE =>
                    read_data1 <= registers(to_integer(unsigned(rs1)));
                    read_data2 <= registers(to_integer(unsigned(rs2)));

```

```

state <= EXECUTE;

-- -----
-- EXECUTE
-- -----
when EXECUTE =>
    state <= MEM;

-- -----
-- MEM
-- -----
when MEM =>
    -- Accès mémoire données (bit 10 = 0)
    if alu_result(10) = '0' then
        if mem_read = '1' then
            dmem_data <= dmem(to_integer(unsigned(alu_result(9 downto 2))));
        end if;
        if mem_write = '1' then
            dmem(to_integer(unsigned(alu_result(9 downto 2)))) <= read_data2;
        end if;
    end if;

    -- Accès GPIO (adresse 0x400 / 0x404, bit 10 = 1)
    if alu_result(10) = '1' then
        if mem_read = '1' and alu_result(10 downto 0) = "1000000000" then
            -- adresse 0x400
            dmem_data <= x"000000" & inport;
        end if;
        if mem_write = '1' and alu_result(10 downto 0) = "10000000100" then
            -- adresse 0x404
            outputport <= read_data2(7 downto 0);
        end if;
    end if;

    state <= WB;

-- -----
-- WRITEBACK
-- -----
when WB =>
    -- Ecriture registre
    if reg_write = '1' and rd /= "00000" then
        if mem_to_reg = '1' then
            registers(to_integer(unsigned(rd))) <= dmem_data;
        else
            registers(to_integer(unsigned(rd))) <= alu_result;
        end if;
    end if;

    -- Mise à jour du PC
    if branch = '1' and zero = '1' then
        PC <= std_logic_vector(unsigned(PC) + unsigned(imm));
    else
        PC <= std_logic_vector(unsigned(PC) + 4);
    end if;

    state <= FETCH;

when TRAP =>
    null;

when others =>
    null;
end case;
end if;
end if;
end process;

-- =====

```

```
-- INITIALISATION (simulation uniquement)
-- =====
-- Note : l'initialisation du banc de registres à zéro
-- et le chargement de imem via $readmemh n'ont pas
-- d'équivalent synthétisable en VHDL standard.
-- Pour la simulation, utiliser un process d'initialisation
-- ou un testbench dédié avec std.textio.

end architecture rtl;
```

Annexe 2

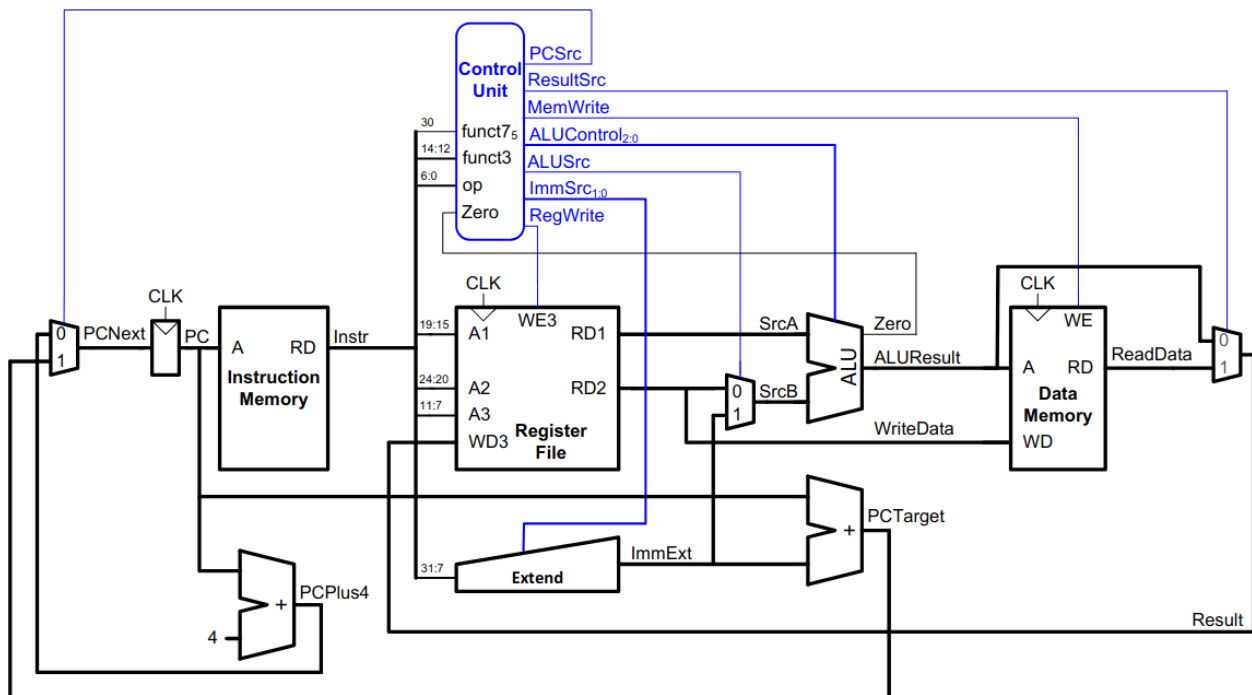
Code source assembleur

https://github.com/JOKleinGe1/Simplified_RISCV/blob/main/src/start.S

```
.section      .text
.global      _start
_start:
    addi x1, x0, 0
    addi x2, x0, 5
    addi x3, x0, 512
    addi x5, x0, 0
loop:
    sw x1, 0(x3)
    lw x4, 0(x3)
    addi x1, x1, 1
    addi x3, x3, 4
    add x5, x5, x4
    beq x1, x2, end
    beq x0, x0, loop
end:
    sub x5, x1, x2
    addi x3, x0, 1024
    lw x4, 0(x3)
    xori x1, x4, 15
    sw x1, 4(x3)
    beq x0, x0, end
```

Chemin de données (datapath), du RISCV (RV32I)

source : <https://user-images.githubusercontent.com/88595269/128730771-560da5b6-f33b-410c-bc03-2dc68f2c748e.png>



.Annexe 3 : RISC-V ISA Reference

Instruction Encoding Templates

	31	25	24-20	19-15	14-12	11-7	6-0
R-type	<i>funct7</i>	<i>rs2</i>				<i>rd</i>	
I-type	<i>imm[11:0]</i>						
S-Type	<i>imm[11:5]</i>		<i>rs1</i>	<i>funct3</i>	<i>imm[4:0]</i>		
B-type	<i>imm[12/10:5]</i>	<i>rs2</i>			<i>imm[4:1/11]</i>	<i>opcode</i>	
J-type		<i>imm[20/10:1/11/19:12]</i>				<i>rd</i>	
U-type		<i>imm[31:12]</i>					

RV32I Base Integer Instruction Set

Arithmetic Instructions

	Instruction	Name	Description	Opcode	Funct3	Funct7
R-type	add rd, rs1, rs2	<i>add</i>	$R[rd] = R[rs1] + R[rs2]$	011 0011	000	000 0000
	sub rd, rs1, rs2	<i>subtract</i>	$R[rd] = R[rs1] - R[rs2]$	011 0011	000	010 0000
	and rd, rs1, rs2	<i>bitwise and</i>	$R[rd] = R[rs1] \& R[rs2]$	011 0011	111	000 0000
	or rd, rs1, rs2	<i>bitwise or</i>	$R[rd] = R[rs1] R[rs2]$	011 0011	110	000 0000
	xor rd, rs1, rs2	<i>bitwise xor</i>	$R[rd] = R[rs1] \wedge R[rs2]$	011 0011	100	000 0000
	sll rd, rs1, rs2	<i>shift left logical</i>	$R[rd] = R[rs1] \ll R[rs2]$	011 0011	001	000 0000
	srl rd, rs1, rs2	<i>shift right logical</i>	$R[rd] = R[rs1] \gg R[rs2]$	011 0011	101	000 0000
	sra rd, rs1, rs2	<i>shift right arithmetic</i>	$R[rd] = R[rs1] \ggg R[rs2]$	011 0011	101	010 0000
	slt rd, rs1, rs2	<i>set less than (signed)</i>	if ($R[rs1] < R[rs2]$) { $R[rd] = 1$; } else { $R[rd] = 0$; }	011 0011	010	000 0000
	sltu rd, rs1, rs2	<i>set less than (unsigned)</i>	if ($R[rs1] < R[rs2]$) { $R[rd] = 1$; } else { $R[rd] = 0$; }	011 0011	011	000 0000
	addi rd, rs1, imm	<i>add immediate</i>	$R[rd] = R[rs1] + \text{imm}(\text{sign-extend imm})$	001 0011	000	n/a
	andi rd, rs1, imm	<i>bitwise and immediate</i>	$R[rd] = R[rs1] \& \text{imm}(\text{sign-extend imm})$	001 0011	111	n/a
	ori rd, rs1, imm	<i>bitwise or immediate</i>	$R[rd] = R[rs1] \text{imm}(\text{sign-extend imm})$	001 0011	110	n/a
	xori rd, rs1, imm	<i>bitwise xor immediate</i>	$R[rd] = R[rs1] \wedge \text{imm}(\text{sign-extend imm})$	001 0011	100	n/a
	slti rd, rs1, imm	<i>set less than (signed) immediate</i>	if ($R[rs1] < \text{imm}$) { $R[rd] = 1$; } else { $R[rd] = 0$; }	001 0011	010	n/a
	sltiu rd, rs1, imm	<i>set less than (unsigned) immediate</i>	if ($R[rs1] < \text{imm}$) { $R[rd] = 1$; } else { $R[rd] = 0$; }	001 0011	011	n/a
I-type	slli rd, rs1, imm	<i>shift left logical immediate</i>	$R[rd] = R[rs1] \ll \text{imm}[4:0]$	001 0011	001	000 0000
	srli rd, rs1, imm	<i>shift right logical immediate</i>	$R[rd] = R[rs1] \gg \text{imm}[4:0]$	001 0011	101	000 0000
	srai rd, rs1, imm	<i>shift right immediate</i>	$R[rd] = R[rs1] \ggg \text{imm}[4:0]$	001 0011	101	010 0000

Instruction	Name	Description	Opcode	Funct3	Funct7
	<i>arithmetic</i>				
	<i>immediate</i>	<i>(sign-extend)</i>			

Memory Instructions

	Instruction	Name	Description	Opcode	Funct3
	lb rd, imm(rs1)	load byte	$R[rd] = M[R[rs1] + imm]$ [7:0] <i>(sign-extend)</i>	000 0011	000
	lbu rd, imm(rs1)	load byte <i>(unsigned)</i>	$R[rd] = M[R[rs1] + imm]$ [7:0] <i>(zero-extend)</i>	000 0011	100
I-type	lh rd, imm(rs1)	load half-word	$R[rd] = M[R[rs1] + imm]$ [15:0] <i>(sign-extend)</i>	000 0011	001
	lhu rd, imm(rs1)	load half-word <i>(unsigned)</i>	$R[rd] = M[R[rs1] + imm]$ [15:0] <i>(zero-extend)</i>	000 0011	101
	lw rd, imm(rs1)	load word	$R[rd] = M[R[rs1] + imm]$ [31:0] <i>(sign-extend in RV64I)</i>	000 0011	010
S-Type	sb rs2, imm(rs1)	store byte	$M[R[rs1] + imm][7:0] =$ $R[rs2][7:0]$	010 0011	000
	sh rs2, imm(rs1)	store half-word	$M[R[rs1] + imm][15:0] =$ $R[rs2][15:0]$	010 0011	001
	sw rs2, imm(rs1)	store word	$M[R[rs1] + imm][31:0] =$ $R[rs2][31:0]$	010 0011	010

Control Instructions

	Instruction	Name	Description	Opcode	Funct3
	beq rs1, rs2, label	branch if equal	if ($R[rs1] ==$ $R[rs2]$) { PC = PC + offset }	110 0011	000
	bne rs1, rs2, label	branch if not equal	if ($R[rs1] !=$ $R[rs2]$) { PC = PC + offset }	110 0011	001
B-type	blt rs1, rs2, label	branch if less than <i>(signed)</i>	if ($R[rs1] <$ $R[rs2]$) { PC = PC + offset }	110 0011	100
	bltu rs1, rs2, label	branch if less than <i>(unsigned)</i>		110 0011	110
	bge rs1, rs2, label	branch if greater or equal <i>(signed)</i>	if ($R[rs1] >=$ $R[rs2]$) { PC = PC + offset }	110 0011	101
	bgeu rs1, rs2, label	branch if greater or equal <i>(unsigned)</i>		110 0011	111
J-type	jal rd, label	jump and link	$R[rd] = PC + 4$ PC = PC + offset	110 1111	n/a
I-type	jalr rd, rs1, label	jump and link register	$R[rd] = PC + 4$ PC = $R[rs1] + imm$	110 0111	000

Other Instructions

	Instruction	Name	Description	Opcode	Funct3
U-type	auipc rd, immu	add upper immediate to PC	$R[rd] = PC + imm <<$ 12 <i>(sign-extend shifted</i> <i>immediate)</i>	011 0111	n/a
	lui rd, immu	load upper immediate	$R[rd] = imm << 12$ <i>(sign-</i> <i>extend shifted immediate)</i>	011 0111	n/a
I-type	ebreak	environment BREAK	asks the debugger to do something (<i>imm=0</i>)	111 0011	000
	ecall	environment ECALL	asks the OS to do something (<i>imm=1</i>)	111 0011	000